



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Project: 24-Bit Multi-Mode Processor

Course Instructor: Prof. Dr. Ghulam Abbas

Course: Computer Organization and Assembly Language - CE222

Faculty: Cyber Security(Section)

Group Members: Muhammad Shariq Usman(2024465), Sohaib Bin Tausif(2024595)

Date of Submission: April 20th, 2026

Abstract:

This report presents the design of a novel 24-bit stored program processor developed for CE222 Computer Organization and Assembly Language. Our design improves upon the Mano Machine baseline across several key areas. The single accumulator is replaced by an eight register general purpose file, the E flip-flop is replaced by a full Process Status Register with condition flags and interrupt control bits, and the instruction set is expanded to 23 instructions adding proper subroutine support through CALL and RET, stack operations through PUSH and POP, and true shift operations through SHL and SHR. Three addressing modes are supported through a dedicated 2-bit mode field allowing for a two level decode scheme that simultaneously extracts all instruction fields in a single clock cycle. The ALU is implemented as three parallel functional units an adder-subtractor, a logic unit and a shift unit feeding a multiplexer controlled by the hardwired Control Unit. Both programmed and interrupt driven I/O are supported.

Table of Contents

I. Stored Program Organization.....	3
II. Memory Organization.....	3
III. Computer Registers.....	4
IV. Addressing Modes.....	5
A. Mode 00 — No Address Mode (NA).....	6
B. Mode 01 — Direct Memory Access (DMA).....	7
C. Mode 10 — Indirect Memory Access (IMA).....	7
V. Instruction Set.....	8
VI. Common Bus System.....	10
VII. ALU Design.....	12
VIII. Control Unit Design and Timing Signals.....	19
IX. Instruction Cycle.....	23
X. Input/Output and Interrupt Handling.....	27
XI. Complete Computer Description.....	30
XII. Bibliography.....	31

I. Stored Program Organization

Our processor is built upon the stored program concept which forms the foundational principle of all general purpose processors. In this model both the program instructions and the data they operate on are stored together in the same unified memory unit. The processor fetches each instruction sequentially from memory using the Program Counter, loads it into the Instruction Register, decodes it through the Control Unit, and then executes it by issuing the appropriate sequence of microoperations. This means our processor does not require separate instruction and data memories a single unified memory of 4096 locations each 24 bits wide serves both purposes simultaneously. The stored program concept is what gives our processor its reprogrammability changing what the processor does simply requires loading different instructions into memory rather than modifying any hardware.

Our processor utilizes a 24-bit instruction format which follows the format shown in Table I below. We allocate 5 bits to the opcode giving us up to 32 possible operations. We also include a 2-bit mode field which determines the addressing mode of the instruction. We have allocated 3 bits to Rd giving us 8 destination registers and 2 bits to Rs giving us 4 source registers. Finally 12 bits are allocated to the address field giving us 4096 addressable memory locations.

TABLE I: 24-BIT INSTRUCTION FORMAT

opcode (5-bit)	mode (2-bit)	Rd (3-bit)	Rs (2-bit)	Address (12-bit)
bits 23–19	bits 18– 17	bits 16– 14	bits 13– 12	bits 11– 0

II. Memory Organization

In our design since we have 12 bits allocated to addresses that gives us a total of 4096 locations to work with and since our processor has a 24-bit word size that gives us $4096 \times 24 = 12,288$ bytes or approximately 12.3 kilobytes of overall memory to use. All memory related tasks in our processor will strictly flow through the Address Register as it alone is the only register with a direct connection to the memory address bus. The LOAD instruction enables read operations by loading the address field of the instruction into the AR and retrieving the 24-bit value stored there and then storing that value in Rd. For write operations our STORE instruction is used the address field is loaded into AR and the value currently held in Rd is then written to that memory location. Our Stack Register (R7) also works with support from the memory as the PUSH and POP instructions read and write to memory addresses pointed to by R7. In mode 00 No Address mode or NA mode no memory access occurs at all as both operands are already present in registers. In mode 01 Direct Memory Access mode or DMA mode one memory access occurs where the address field is loaded into AR and the value at that location is retrieved. In mode 10 Indirect Memory Access mode or IMA mode two memory accesses are required the first retrieves the

effective address and the second retrieves the actual operand. The stack is also memory backed meaning PUSH and POP operations read and write to memory addresses pointed to by R7, all of which also route through AR.

III. Computer Registers

TABLE II: REGISTER ORGANIZATION

Register Name	Acronym	Size	Purpose
Program Counter	PC	12-bit	Holds address of next instruction to fetch
Zero Register	ZR (R0)	24-bit	Permanently holds value 0, read only
General Purpose Registers	R1-R5	24-bit each	Free registers for operands and results
Link Register	LR (R6)	24-bit	Saves return address during CALL
Stack Register	SR (R7)	24-bit	Points to top of stack, initialized at 0xFFFF
Address Register	AR	12-bit	Only register connected to memory address bus
Instruction Register	IR	24-bit	Holds currently fetched instruction for decoding
Process Status Register	PSR	8-bit	Holds processor condition flags and control bits

In our design the first register that we are working with is the Program Counter or PC register which we will use to keep track of our operations by holding the address of the next instruction to be fetched from the memory and will operate in a sequential sequence where it will increment from instruction to instruction based on which instructions were stored in the memory first barring CALL or any branching instructions where the address section of the branch instruction will be transferred to the PC to act as the address for the next instruction. We also have a standard Instruction Register which will store the current instruction while the Control Unit reads and decodes it. The IR feeds its fields directly to the CU decode logic through dedicated internal lines bits 20-23 go to the opcode decoder, bits 18-19 to the mode decoder, bits 15-17 to the Rd decoder, bits 12-14 to the Rs decoder or extended opcode logic, and bits 0-11 directly to the AR.

The next registers we will talk about are a series of general purpose registers which go from R0 to R7. R0 will be our zero register or ZR which cannot be written to and will constantly hold the 0 operand allowing us to perform operations such as clear operations. R1-R5 will be fully free for the processor to use and will hold the operands

on which the ALU will perform arithmetic computations. R6 is our Link Register or LR this will allow us to implement CALL functionality into our processor by saving the return address every time a CALL instruction activates. At the end of the function the RET instruction will execute and take the address from the LR and put it in the PC. R7 will be our Stack Register or SR and will help enable PUSH, POP, CALL and RET functionality. It will be initialized at the start of the program at 0xFFFF which is equivalent to address 4095 in decimal the highest address available in our 12-bit address space as our stack grows downwards.

The Address Register or AR has 12 bits to accommodate the width of the memory address and is the only register with a direct connection to the memory address bus. Next we have the Process Status Register or PSR. It is not connected to the general bus instead it has direct connections to the ALU to capture the Z, N, C and V condition flags. The Z flag is set when the result of an operation is 0. The N flag is set when the result is negative indicated by the MSB being 1. The C flag is the carry flag which handles carry outputs during arithmetic operations. The V flag is the overflow flag which indicates signed overflow during ADD, SUB and INC by checking whether two operands of the same sign produced an operand with a different sign. The PSR also contains an I bit which enables or disables interrupt handling.

IV. Addressing Modes

TABLE III: ADDRESSING MODE COMPARISON

Our processor supports three addressing modes determined by the 2-bit mode field present in every instruction. Each mode defines a different method by which the processor resolves the location of an operand.

Mode	Name	Memory Accesses	Speed	Used By
00	No Address (NA)	0	Fastest	ADD, SUB, AND, OR, XOR, NOT, MOV, CIL, CIR
01	Direct Memory Access (DMA)	1	Medium	LOAD, STORE
10	Indirect Memory Access (IMA)	2	Slowest	LOAD, STORE with pointer

A. Mode 00 — No Address Mode (NA)

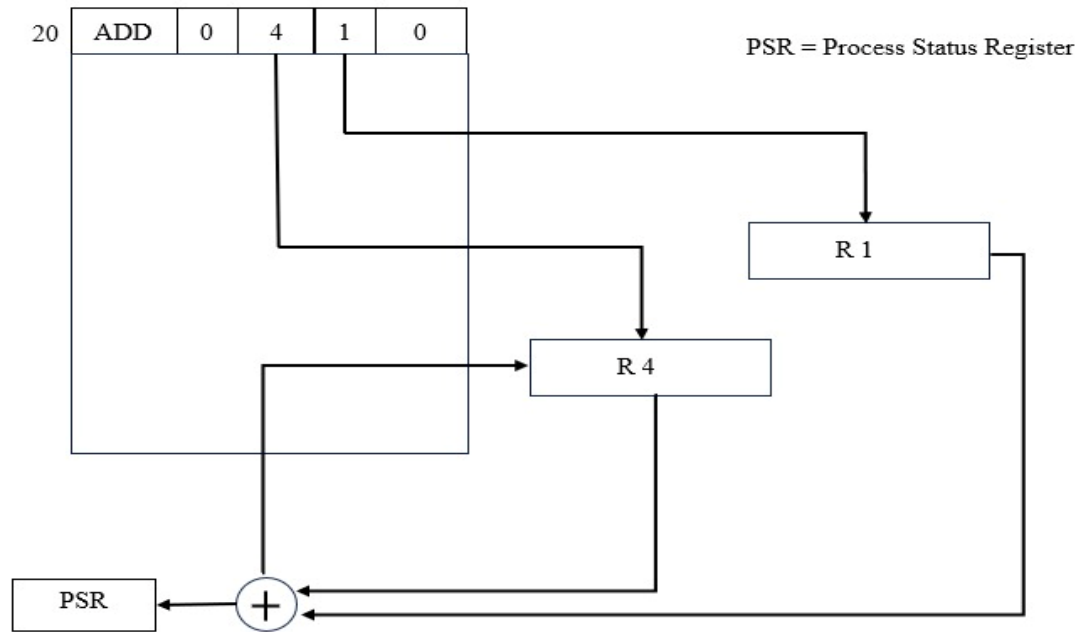


FIGURE 1:NA

When the mode is set to 00 the ALU will directly read values from the Rs and Rd registers and write all results back to Rd. This is the fastest mode as all latency which would have occurred due to repeatedly accessing memory is no longer present. Register operations taking advantage of this mode are ADD, SUB, AND, OR, XOR, NOT, MOV, CIL and CIR. As an illustration, consider an ADD instruction stored in memory at address 100. The mode bits are 00 so the processor enters NA mode. Rd contains R2 which holds the value 15 and Rs contains R3 which holds the value 10. Since both operands are already present in the registers the ALU directly performs the addition and stores the result 25 back into R2. No memory access is required at any point during this operation.

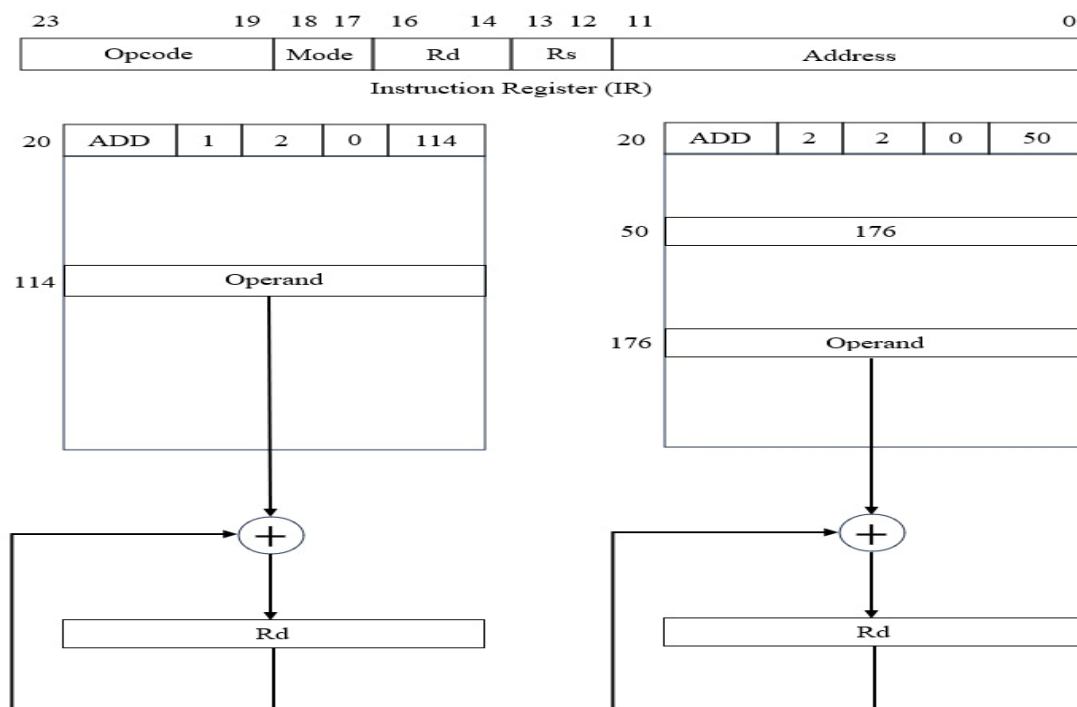


FIGURE 2: DMA and IMA

B. Mode 01 — Direct Memory Access Mode (DMA)

When mode is set to 01 the processor enters DMA mode and the 12-bit address field of the instruction contains the exact memory address where the operand is located. The processor loads this address into AR and performs a single memory access to retrieve or store the operand. This requires one memory access and is therefore slower than mode 00 but faster than mode 10. Our LOAD and STORE instructions operate in this mode. As an illustration, consider a LOAD instruction stored in memory at address 200. The mode bits are 01 so the processor enters DMA mode. The address field contains the binary equivalent of 500. The processor loads 500 into AR and performs a single memory access to retrieve the 24-bit value stored at address 500. That value is then transferred into the destination register Rd. The effective address of the operand in this instruction is 500.

C. Mode 10 — Indirect Memory Access Mode (IMA)

When mode is set to 10 the processor enters IMA mode and the address in the address field will not directly point to the operand but will instead point to a memory location which contains the actual address of the operand. This is the slowest mode as memory must be accessed twice. However it provides the advantage of dynamic address resolution a single instruction can point to different values throughout the same program by modifying the pointer address stored in memory. Consider a LOAD instruction stored in memory at address 300. The mode bits are 10 so the processor enters IMA mode. The address field contains the binary equivalent of 600. The processor first loads 600 into AR and performs a memory access to retrieve the value stored at address 600. The value found at address 600 is not the operand itself but rather another address 1200. The processor then loads 1200 into AR and performs a second memory access to retrieve the actual operand stored there. The effective address of the operand in this instruction is 1200.

V. Instruction Set

Our processor implements 23 instructions covering all four required categories: arithmetic and logic, data transfer, program control, and input/output. Table V presents the complete instruction set with hexadecimal encodings and descriptions.

TABLE IV: COMPLETE INSTRUCTION SET

Symbol	I=0	I=1	Description
ADD	1xxx	9xxx	Add operand in Rs to Rd, store result in Rd, update PSR flags
SUB	2000	A000	Subtract Rs from Rd, store result in Rd, update PSR flags
AND	0xxx	8xxx	Bitwise AND between Rd and Rs, store result in Rd, update PSR flags
LOAD	5xxx	Bxxx	Load 24-bit value from memory address into destination register Rd
STORE	3xxx	Cxxx	Store value in register Rd into memory at specified address
OR	7300		Bitwise OR between Rd and Rs, store result in Rd, update PSR flags
XOR	7500		Bitwise XOR between Rd and Rs, store result in Rd, update PSR flags
NOT	7200		Bitwise complement of Rd, store result in Rd, update N and Z flags
CIL	7600		Rotate Rd left by one bit, leftmost bit wraps to rightmost, update C flag
CIR	7680		Rotate Rd right by one bit, rightmost bit wraps to leftmost, update C flag
SHL	7008		Shift Rd left by one bit, rightmost position filled with 0, leftmost bit discarded
SHR	7010		Shift Rd right by one bit, leftmost position filled with 0, rightmost bit discarded
INC	7020		Increment value in Rd by 1, update PSR flags
MOV	7900		Copy value from Rs into Rd, no memory access
JMP	7700		Unconditional jump to specified address, load address into PC
JZ	7701		Jump to specified address if Z flag in PSR is set, otherwise continue
JN	7702		Jump to specified address if N flag in PSR is set, otherwise continue

JC	7703		Jump to specified address if C flag in PSR is set, otherwise continue
CALL	7780		Push current PC onto stack via R7, decrement R7, jump to address
RET	77C0		Pop return address from stack into PC, increment R7
PUSH	7F00		Decrement R7, store value of Rd at memory address pointed to by R7
POP	7F80		Load value from memory address pointed to by R7 into Rd, increment R7
HLT	7001		Halt processor execution, clear I bit in PSR to disable interrupts
ION	7040		Set PSR[I] to 1, enable interrupts
IOF	7080		Clear PSR[I] to 0, disable interrupts

VI. Common Bus System

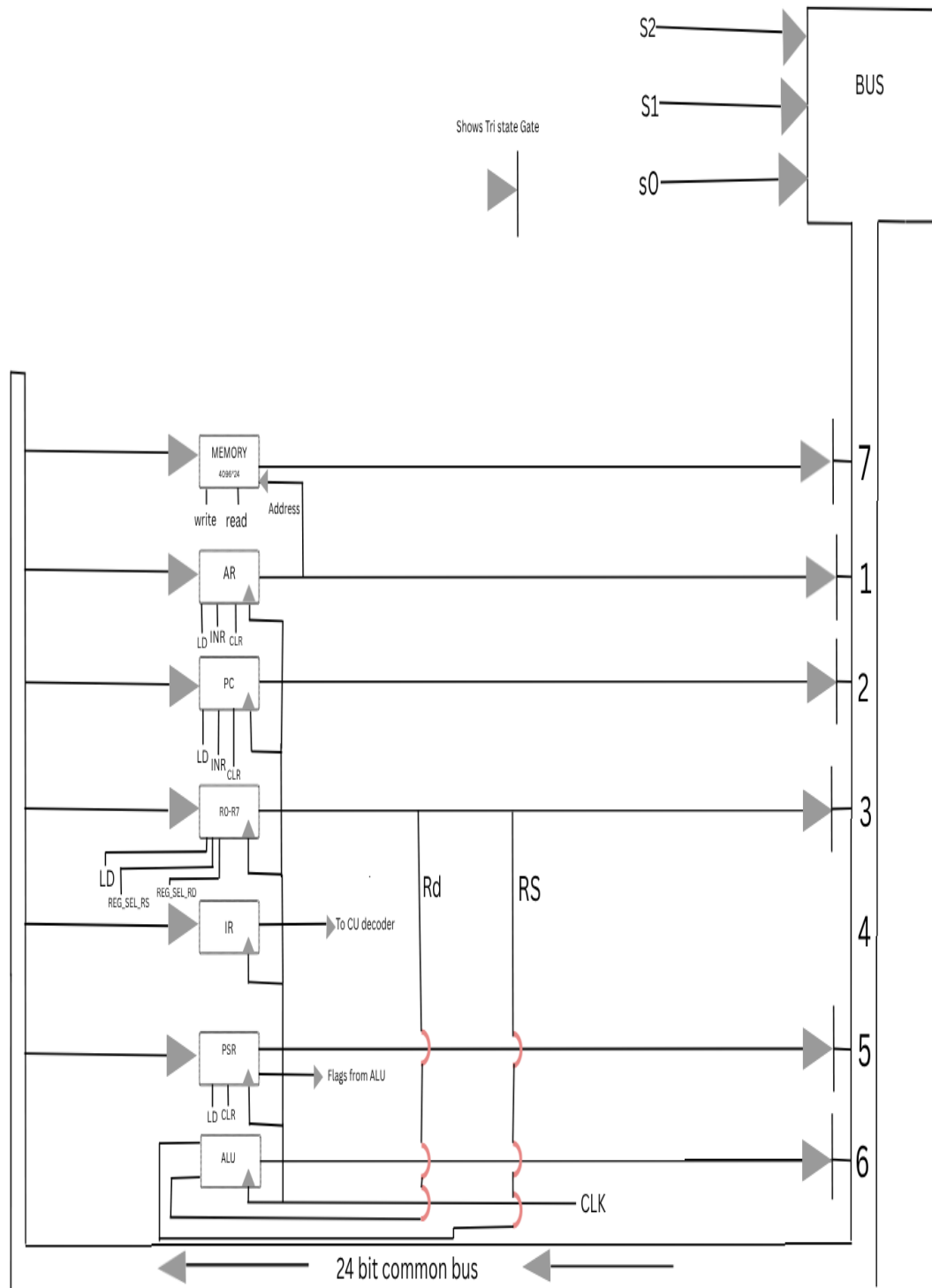


FIGURE 3: CONTROL BUS

TABLE V: BUS SELECTION VARIABLES

S2	S1	S0	Decimal	Component Enabled
0	0	0	0	None (bus idle)
0	0	1	1	AR (Address Register)
0	1	0	2	PC (Program Counter)
0	1	1	3	R0-R7 (Register File)
1	0	0	4	R (feeds CU directly no bus output)
1	0	1	5	PSR (Process Status Register)
1	1	0	6	ALU output
1	1	1	7	Memory output

Our processor utilizes a tri-state 24-bit common bus system built on the tri-state gate buffer which works by acting as a type of current valve that can shut off the output from specific components whenever the select signal going to that specific component is inactive. The tri-state gate buffer has three modes: logic high, logic low, and high impedance, where the high impedance state effectively disconnects the component from the bus despite the component remaining physically wired to it. The system is controlled by three selection variables S0, S1 and S2 which are connected to the Control Unit. During different clock cycles the values of S0, S1 and S2 determine which register is active on the bus and which registers are in high impedance mode.

The bus connects seven components — Memory (4096x24), AR, PC, R0-R7, IR, PSR, and ALU — each assigned a unique selection number. IR is a notable exception: in our design the address field of IR is directly connected to the AR through dedicated internal control lines within the Control Unit, eliminating the need for IR to drive the common bus and simplifying the bus design. Additionally, the AR maintains a dedicated address line directly connected to the memory unit separate from the common bus, as memory addressing must always be available independently of whatever data transfer is currently occurring on the bus. Each component with the exception of IR is assigned a number 1-7 as shown in the diagram, and whenever S0, S1, S2 form a binary combination whose decimal equivalent matches one of these numbers, the corresponding component is enabled on the bus.

VII. ALU Design

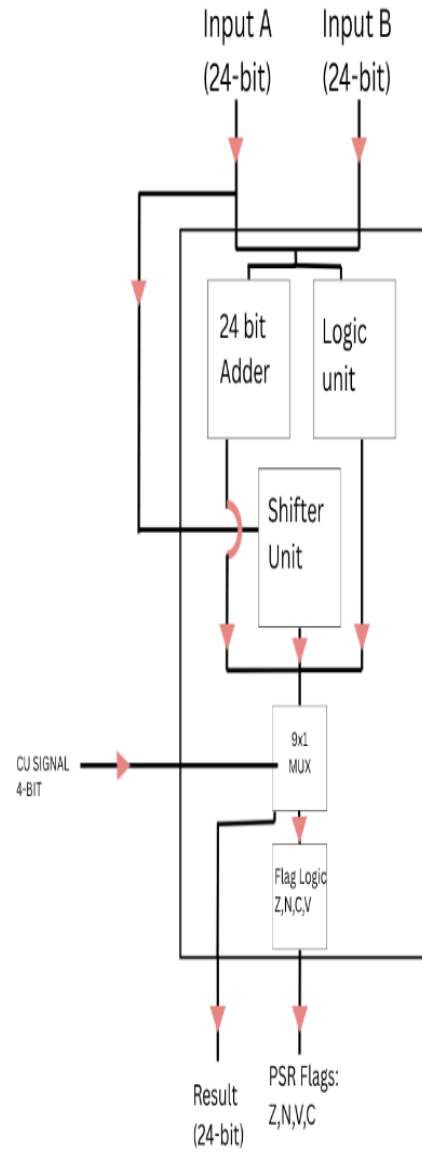


FIGURE 4: ALU DESIGN OVERVIEW

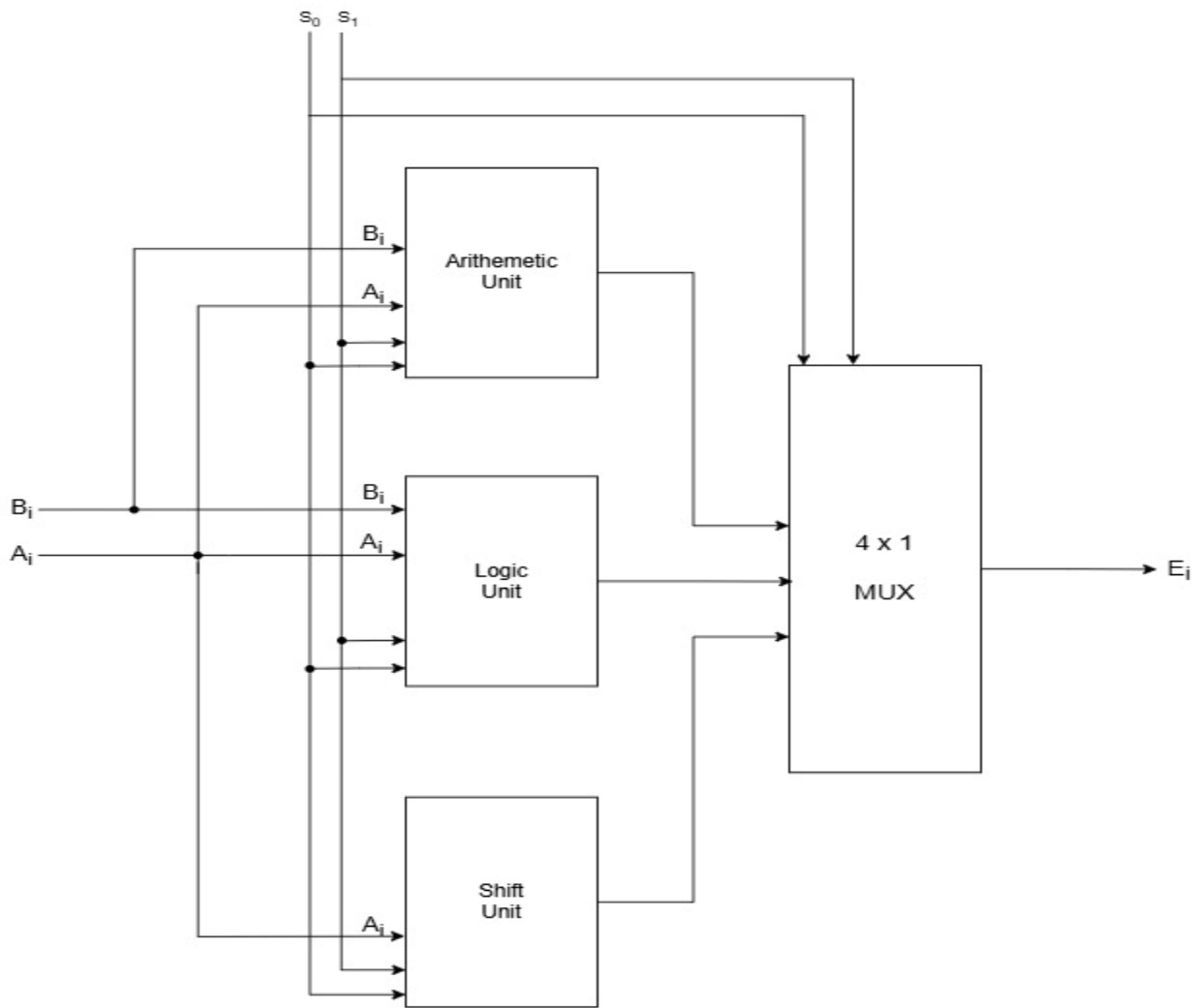


FIGURE 5: ALU DESIGN OVERVIEW 2

TABLE V:CU SELECTION VARIABLES

CU SIGNAL (binary)	Operation
0000	INC
0001	ADD
0010	SUB
0011	AND
0100	OR
0101	XOR
0110	NOT
0111	CIL
1000	CIR
1001	SHL
1010	SHR

Explanation:

Our ALU is a 24-bit circuit that accepts two input operands one of which is INPUT A which is from Rd and the other is INPUT B which is from Rs. The ALU receives a 4 bit operation select signal from the CU which determines which operation is performed.

In total 11 operations can be performed which are ADD,SUB,AND,OR,XOR,NOT,CIL,CIR,SHL and SHR. Internally speaking the ALU has three units which consists of the 24-bit Adder which handles all of the arithmetic operations, the second is the logic unit which handles the bitwise operations and the third is shifter which handles the CIL,CIR,SHL and SHR operations. All three units are connected to a single 9-1 Multiplexer as we are managing 8 general purpose registers and 1 memory unit. From the Multiplexer there are two outputs the first one being the 24-bit result which is written back into Rd and the second is fed into the Flag Logic unit. The Flag Logic unit evaluates four conditions which we discussed in the computer registers section based on the selected result and updates the corresponding bits of the PSR through dedicated direct connections that bypass the common bus entirely.

24-bit Adder Subtractor:

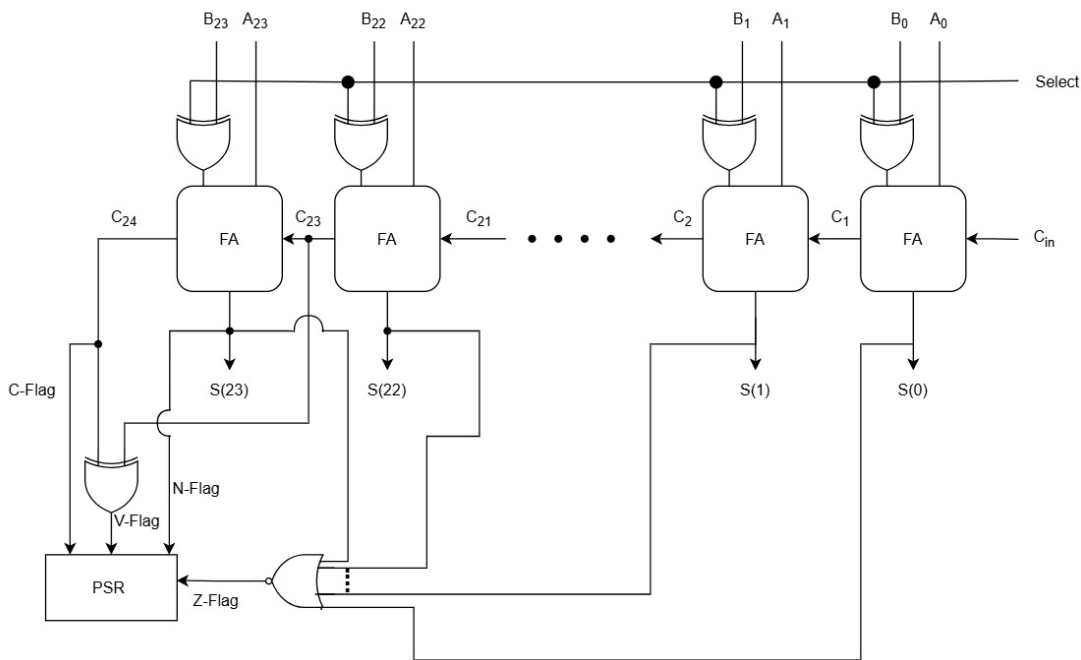


FIGURE 6: 24-bit Adder Subtractor Circuit

TABLE V1: ADDER SELECT SIGNAL

SELECT	CIN	OPERATION	EXPRESSION
0	0	Addition	$R_d + R_s$
1	1	Subtraction	$R_d + \sim R_s + 1 = R_d - R_s$

Arithmetic Circuit Overview

Explanation:

The Adder carries out addition operations by taking inputs A and B and sending them through a 24 bit ripple carry adder with each bit having calculations for a sum and carry bit being done on it. Eventually the carry bit from bit position 23 will go into the C flag of our PSR. If when comparing the carry out of bit 23 with the carry into bit 23 these two carry values differ, signed overflow has occurred and the V flag is set to 1

The Adder carries out subtraction by bitwise inverting input B and then setting its carry input to 1 which is equivalent to inverting and incrementing it.

For increment Input B is hardwired to zero and carry input is set to 1.

Logic Unit:

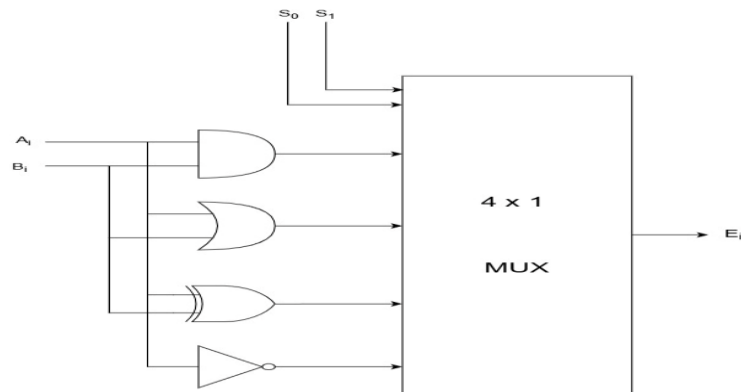


FIGURE 7: Logic Circuit

TABLE V11: LOGIC CIRCUIT SELECT SIGNAL

S1	S0	Operation
0	0	AND
0	1	OR
1	0	XOR
1	1	NOT

The logic unit treats each bit in the 24-bit A and B inputs separately when applying bitwise logic operations to them. Every Logic operation with the exception of NOT works by having 24 gates each being fed one bit from out 24-bit inputs from A and B while the NOT logic operations is carried out by having 24 NOT gates which are only fed inputs from INPUT A.

SHIFT Unit

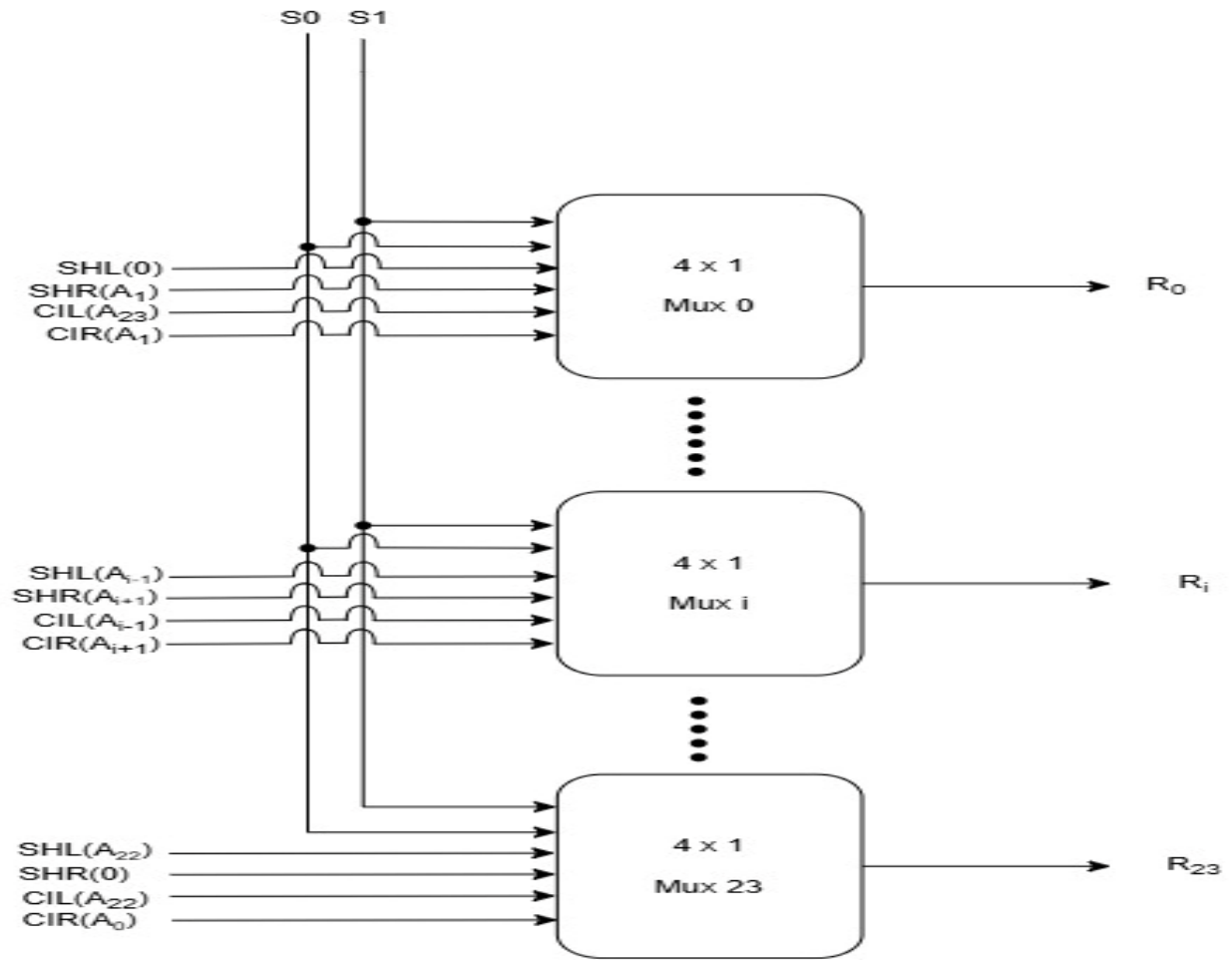


FIGURE 8: SHIFT UNIT CIRCUIT

TABLE V1: SHIFT UNIT SELECT SIGNALS

S1	S0	Operation Selected
0	0	SHL
0	1	SHR
1	0	CIL
1	1	CIR

The shift unit will be used to perform our CIL,CIR,SHL and SHR operations. It will only perform operations on INPUT A since our shift functions will act on a single operand.

In CIL all bits will move one position to the left and the bit that was at bit position 23 will wrap around to bit position 0.

In CIR all bits rotate one position to the right and the bit at bit position 0 will wrap around to bit position 23.

In SHR all bits will move one position to the right and the bit in position 0 will be discarded and replaced with 0.

In SHL all bits will move one position to the left and the bit in position 23 will be discarded and replaced with 0.

VIII. Control Unit Design

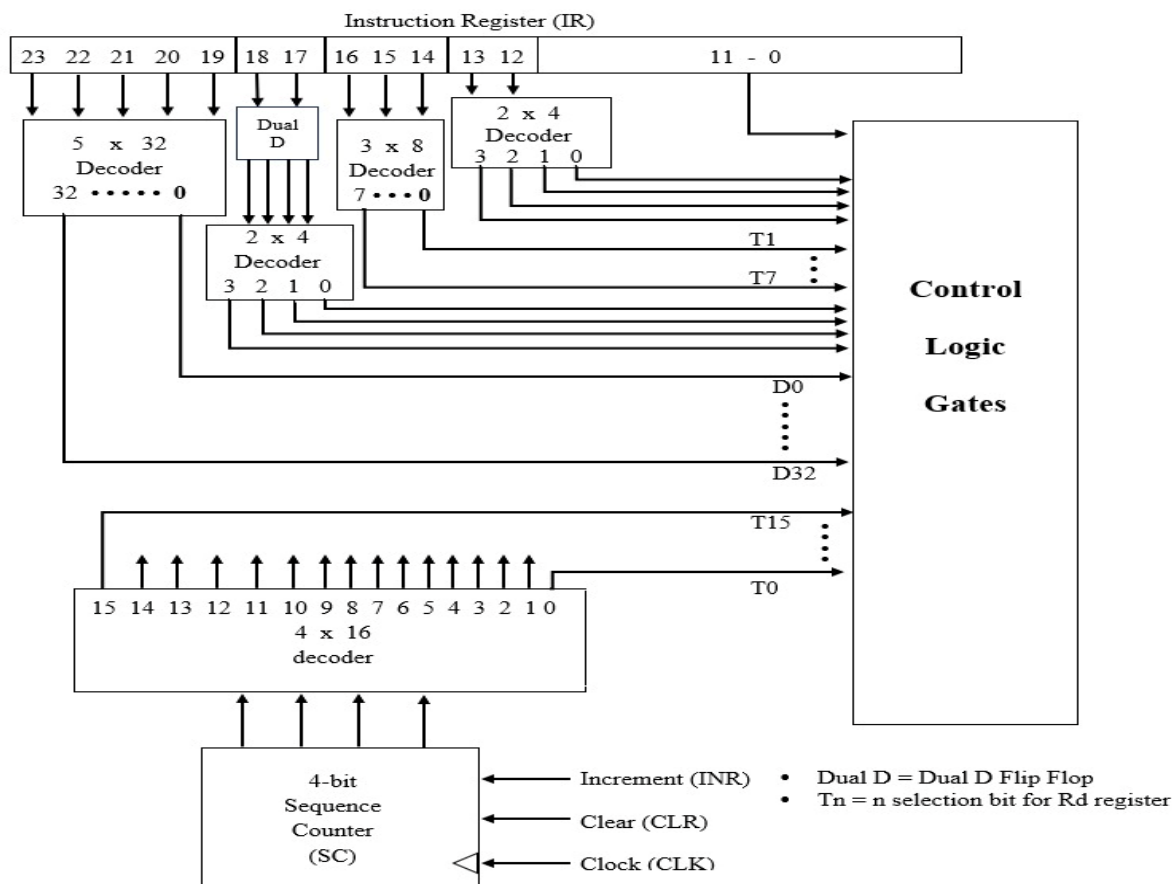


FIGURE 9: CONTROL UNIT DESIGN OVERVIEW

Our processor utilizes a hardwired Control Unit structure meaning all control logic is implemented directly through combinational and sequential logic circuits rather than micro programmed control since the execution is faster as the control signals are directly produced by the Control Unit so there is no need to constantly look up the memory allowing the processor to complete each step in a single clock cycle. This does have a disadvantage that it is less flexible after implementation however this will not matter to us as our instruction set is fixed.

Components of Control Unit:

Our Control Unit has 8 main subsystems which are the Sequence Counter(SC), Control Logic Gates, Instruction Decoder(5x32), Mode Decoder(2x4), Dual D flip flop, Register Decoders (3x8) and (2x4), the Timing Decoder(4x16).

1.Sequence Counter(SC):

This is a 4 bit binary counter that increments itself every clock pulse. Its connected to the Timing Decoder which produces 16 output timing lines labelled T0-T15. During any clock cycle one of these lines is active and represents the current step being run in the execution cycle. The SC itself possess 3 controlling inputs which are the following:

CLR: This resets the SC back to 0.

INR: This increments the SC every clock pulse.

CLK: This ensures all operations are synchronized with the rest of the processor.

T0-T2 are universally used for fetch and decoded phases. T3 depends on what the instruction requires the processor to do but

Register Mode instructions can occur here and memory mode instructions may run through T4-T6 depending on how many times the memory needs to be accessed.

2.Control Logic Gates:

Primary decision making circuit in the control unit and receives all inputs simultaneously. Receives D0-D31 from the Instruction Decoder as well as M0-M3 Mode decoder outputs, the T0-T15 timing signals which are all combined using AND,NOT and OR gates to produce control signals which drive all 21 control bus signals. For example The control signal that enables the ALU to perform ADD is generated by D0 AND M0 AND T3 all being active simultaneously.

3. Instruction Decoder(5x32):

Bit positions 23-19 are inserted into this decoder which produces 32 unique output lines labelled D0-D31, each corresponding to a different instruction and will be connected to the Control Logic Gates. Depending on current instruction that needs to be run it's assigned output line will be active.

4.Mode Decoder(2x4) + Dual D Flip Flop:

Bits position 18 to 17 carry the 2-bit mode field and connect to the Dual D flip flop which latches the two mode bits on the clock edge this allows the mode bits to remain stable during the T2 phase when the instruction inside the IR is actively being decoded which could last until the T6 phase depending on the instruction. The latched outputs are connected to the Mode Decoder which produces 4 output lines labelled M0-M3. 3 of which are to the three modes we discussed in the addressing modes section. This is then as stated before connected to the Control Logic Gates.

5.Registers Decoders(3x8 +2x4):

Bits positions 16-14 carry the 3-bit Rd field and feed into a 3×8 decoder producing 8 output lines labelled 0 through 7, one for each of the eight destination registers R0 through R7. These outputs tell the register file which register to write the result into after an operation is fully executed. Bits positions 13-12 carry the 2-bit Rs field and feed into a 2×4 decoder producing 4 output lines labelled 0 through 3, one for each of the four available source registers R0 through R3. These outputs tell the register file which register to read the source operand from. The address register bits of 0-11 are directly sent to the AR since they just represent a simple address that requires no decoding.

6.Timing Decoder(4x16):

The Timing Decoder is a 4×16 decoder that receives the 4-bit output of the Sequence Counter and converts it into 16 individual timing signals labeled T0 through T15. At any given clock cycle exactly one of these timing signals is active, representing the current step in the instruction execution cycle. The decoder ensures that only one timing signal is ever high at a time, preventing any two execution steps from occurring simultaneously which could cause data corruption or incorrect control signal generation.

Timing Signals:

To how the SC, Timing Decoder and Control Logic Gates work together I have taken this diagram from Mano and Ciletti, Computer System Architecture, Figure 5-7

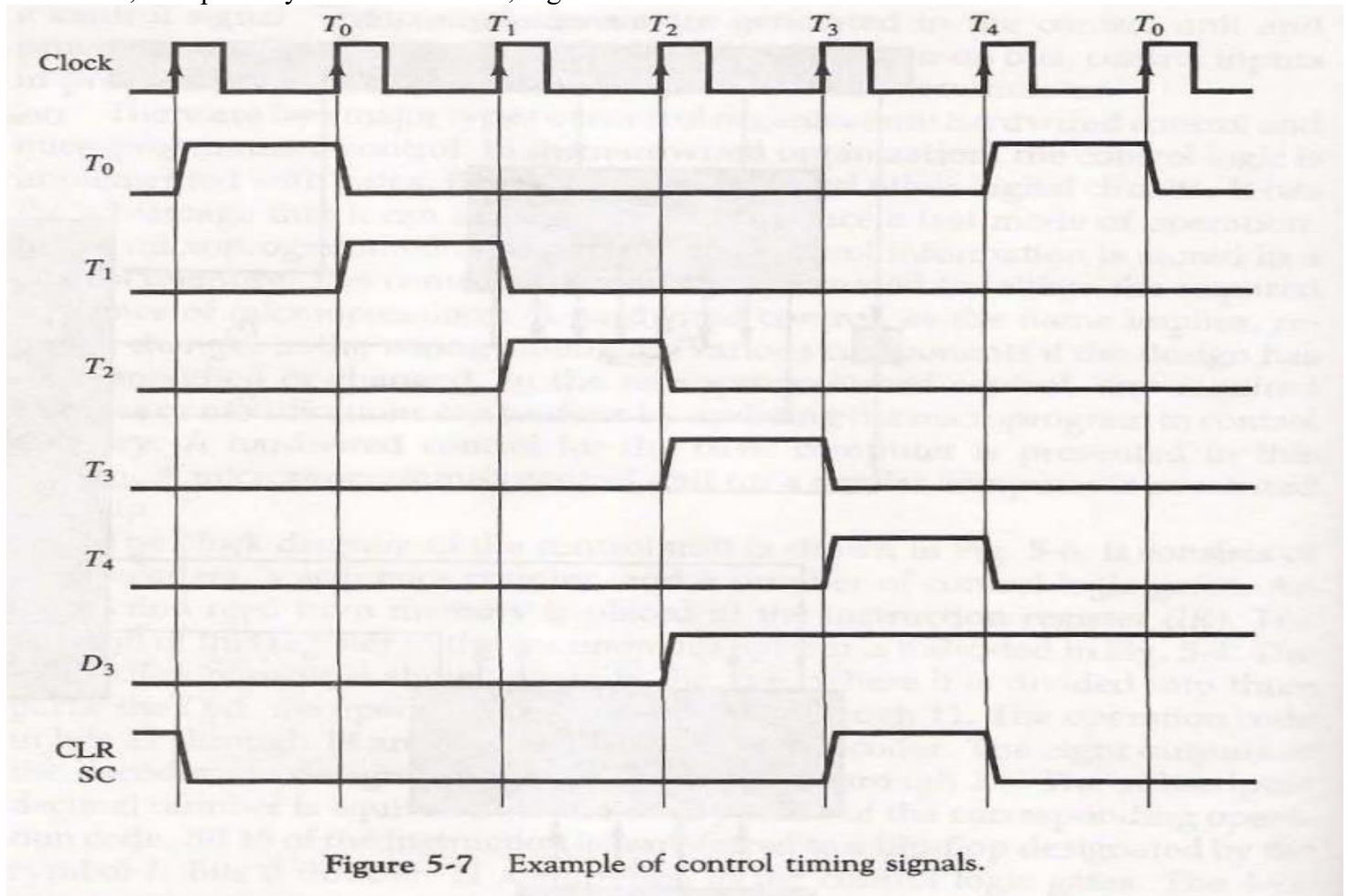


FIGURE 10: TIMING DIAGRAM [1]

Explanation:

The diagram shows the clock signal driving the Sequence Counter which in turn activates timing signals T₀ through T₄ sequentially, one per clock cycle. As shown each timing signal goes high for exactly one clock cycle before the next one activates. The decoder output D₃ representing the STORE instruction goes high at T₂ when the instruction has been fully decoded by the Instruction Decoder. D₃ remains high through T₃ and T₄ as the execute phase runs STORE requires two timing cycles since it must first load the address into AR at T₃ and then write the register value to memory at T₄. Once execution completes at T₄ the CLR SC signal is activated, resetting the Sequence Counter back to zero so T₀ activates again and the next instruction fetch begins immediately.

IX. Instruction Cycle

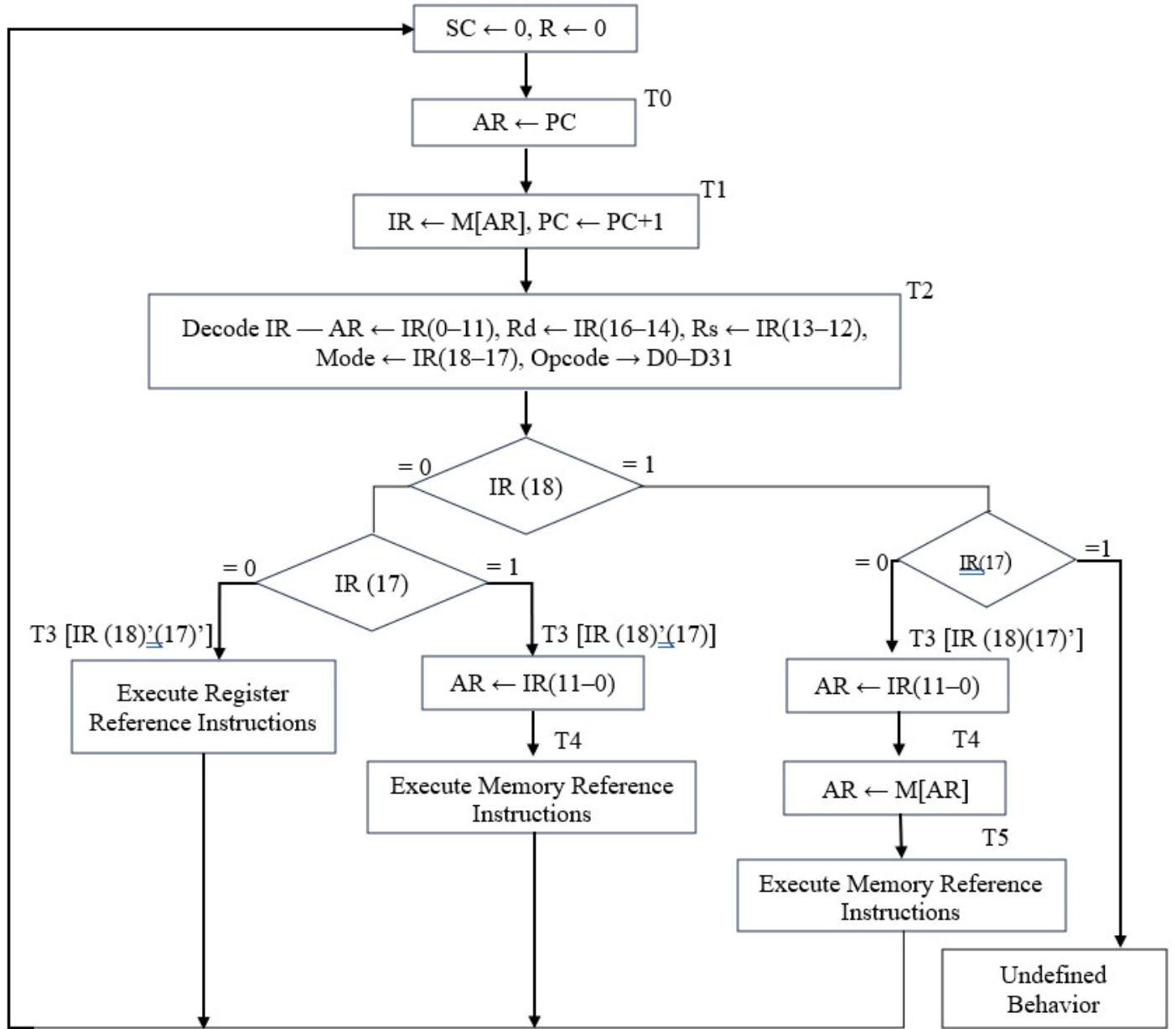


FIGURE 11: Instruction Cycle

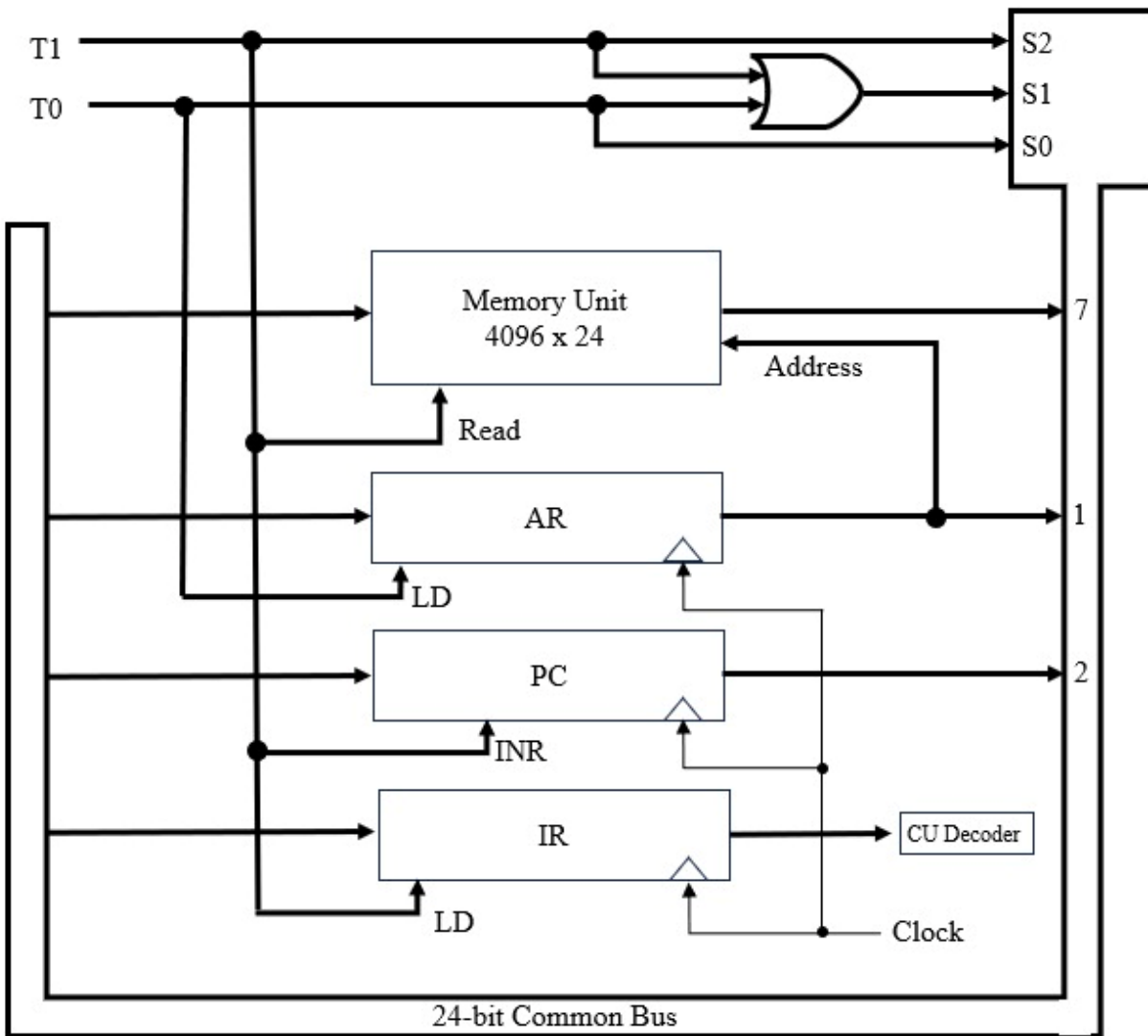


Figure 12: Fetch & Decode with Common Bus

Explanation

All instructions which are run in the processor go through the Fetch, Decode, Execute cycle. The Fetch and Decode phases of this will be exactly the same for every instruction and run from T0 to T2. The Execute phase begins at T3 and is different depending on what instruction is being executed. Once the instruction has been executed the Sequence Counter is reset to 0 and the next cycle begins.

Fetch Phase:

T0: AR ← PC

T1: IR ← M[AR], PC ← PC + 1

Here the address of the next instruction is first loaded into the AR at T0 after which the instruction is loaded into IR and PC is incremented.

Decode Phase:

T2: D0...D31 $\leftarrow$ Decode IR(23-19)
 M0,M1,M2,M3 $\leftarrow$ Decode IR(18-17)
 Rd $\leftarrow$ IR(16-14)
 Rs $\leftarrow$ IR(13-12)
 AR $\leftarrow$ IR(11-0)

Here all decoding happens simultaneously at T2 and the processor will find out which instruction is being executed, which mode it is in, which registers are involved and what addresses to use if any are required provided the mode is not NA mode.

Execute Phase:

Arithmetic and Logic Instructions (Mode 00):

1.ADD

T3: Rd $\leftarrow$ Rd + Rs, PSR $\leftarrow$ Flags(Rd + Rs), SC $\leftarrow$ 0

2.SUB

T3: Rd $\leftarrow$ Rd + $\sim$ Rs + 1, PSR $\leftarrow$ Flags(Rd - Rs), SC $\leftarrow$ 0

3.AND

T3: Rd $\leftarrow$ Rd $\wedge$ Rs, PSR $\leftarrow$ Flags(Rd $\wedge$ Rs), SC $\leftarrow$ 0

4.OR

T3: Rd $\leftarrow$ Rd $\vee$ Rs, PSR $\leftarrow$ Flags(Rd $\vee$ Rs), SC $\leftarrow$ 0

5.XOR

T3: Rd $\leftarrow$ Rd $\oplus$ Rs, PSR $\leftarrow$ Flags(Rd $\oplus$ Rs), SC $\leftarrow$ 0

6. NOT

T3: Rd $\leftarrow$ $\sim$ Rd, PSR[N,Z] $\leftarrow$ Flags($\sim$ Rd), SC $\leftarrow$ 0

7. INC

T3: Rd $\leftarrow$ Rd + 1, PSR $\leftarrow$ Flags(Rd + 1), SC $\leftarrow$ 0

8.MOV

T3: Rd $\leftarrow$ Rs, SC $\leftarrow$ 0

Shift and Rotate Instructions (Mode 00):

1.CIL

T3: Rd $\leftarrow$ shl Rd, Rd(0) $\leftarrow$ Rd(23), SC $\leftarrow$ 0

2. CIR

T3: Rd $\leftarrow$ shr Rd, Rd(23) $\leftarrow$ Rd(0), SC $\leftarrow$ 0

3.SHL

T3: Rd $\leftarrow$ shl Rd, Rd(0) $\leftarrow$ 0, SC $\leftarrow$ 0

4.SHR

T3: $Rd \leftarrow \text{shr } Rd, Rd(23) \leftarrow 0, SC \leftarrow 0$

Data Transfer Instructions:

1. STORE (Mode 01)

T3: $AR \leftarrow IR(11-0)$

T4: $M[AR] \leftarrow Rd, SC \leftarrow 0$

2.LOAD (Mode 10)

T3: $AR \leftarrow IR(11-0)$

T4: $AR \leftarrow M[AR]$

T5: $Rd \leftarrow M[AR], SC \leftarrow 0$

3.STORE(Mode 10)

T3: $AR \leftarrow IR(11-0)$

T4: $AR \leftarrow M[AR]$

T5: $M[AR] \leftarrow Rd, SC \leftarrow 0$

4.LOAD(Mode 01)

T3: $AR \leftarrow IR(11-0)$

T4: $Rd \leftarrow M[AR], SC \leftarrow 0$

Program Control Instructions:

1.JMP

T3: $PC \leftarrow IR(11-0), SC \leftarrow 0$

2. JZ

T3: if $PSR[Z] = 1$ then $PC \leftarrow IR(11-0), SC \leftarrow 0$

3.JN

T3: if $PSR[N] = 1$ then $PC \leftarrow IR(11-0), SC \leftarrow 0$

4. JC

T3: if $PSR[C] = 1$ then $PC \leftarrow IR(11-0), SC \leftarrow 0$

5.CALL

T3: $AR \leftarrow R7 - 1, R7 \leftarrow R7 - 1$

T4: $M[AR] \leftarrow PC$

T5: $PC \leftarrow IR(11-0), SC \leftarrow 0$

6. RET

T3: $AR \leftarrow R7$

T4: $PC \leftarrow M[AR], R7 \leftarrow R7 + 1, SC \leftarrow 0$

7. PUSH

T3: $AR \leftarrow R7 - 1, R7 \leftarrow R7 - 1$

T4: $M[AR] \leftarrow Rd, SC \leftarrow 0$

8. POP

T3: $AR \leftarrow R7$

T4: $Rd \leftarrow M[AR], R7 \leftarrow R7 + 1, SC \leftarrow 0$

I/O Instructions:

1. IN

T3: $Rd \leftarrow M[AR1], SC \leftarrow 0$

2. OUT

T3: $M[AR2] \leftarrow Rs, SC \leftarrow 0$

3. HLT

T3: $PSR[I] \leftarrow 0, S \leftarrow 0$

4. ION

T3: $PSR[I] \leftarrow 1, SC \leftarrow 0$

5. IOF

T3: $PSR[I] \leftarrow 0, SC \leftarrow 0$

X. Input/Output and Interrupt Handling

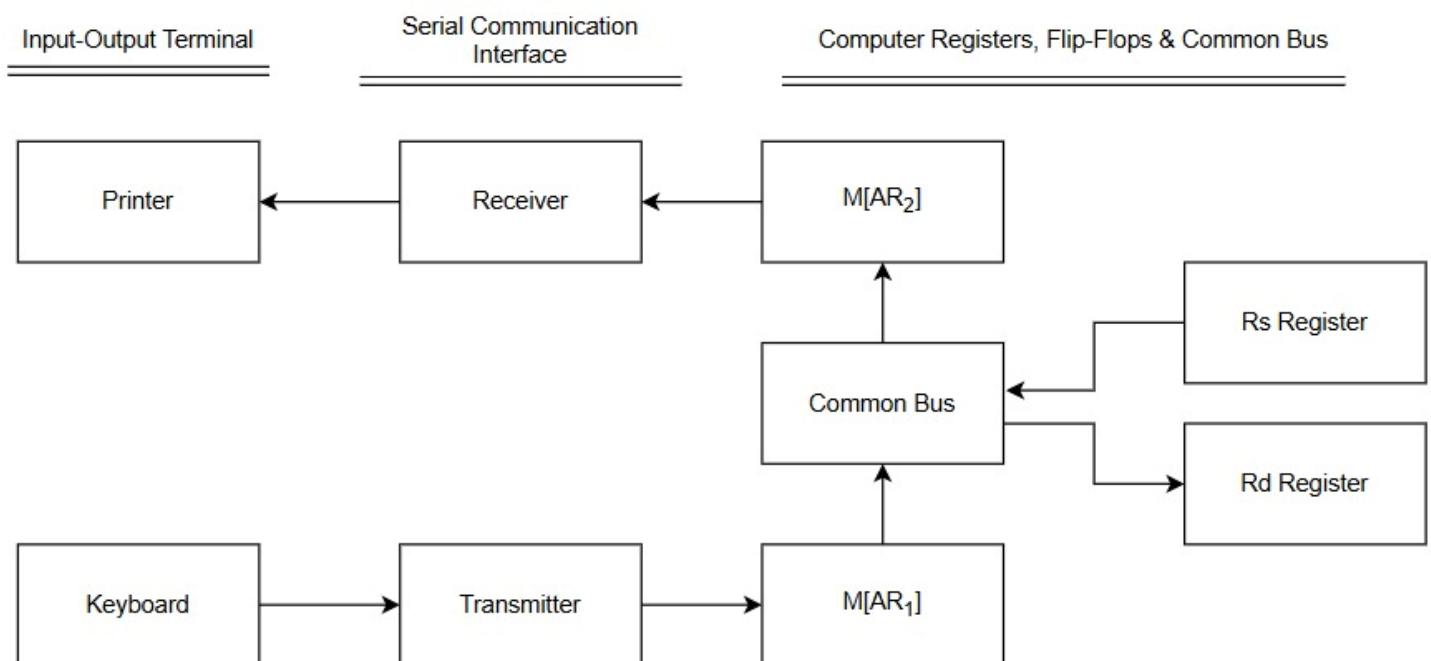


FIGURE 13: Processor Input/Output Configuration

Our process makes use of two approaches of handling input and output operations which are programmed I/O and interrupt driven I/O. In programmed I/O the CPU cannot do anything else while facilitating I/O request until the entire request is finished while in interrupt driven I/O the CPU can continue executing other instructions and only tend to the I/O devices when they send a signal requiring immediate attention. Both methods are facilitated in our design through interrupts which come from the PSR interrupt enable bit. M[AR1] ensures that the data which is fetched is coming as an input and M[AR2] ensures that it is the data for output

A. Programmed I/O:

IN: T3: $Rd \leftarrow M[AR1]$, $SC \leftarrow 0$

OUT: T3: $M[AR2] \leftarrow Rs$, $SC \leftarrow 0$

Explanation:

The CPU directly controls all data transfers between the processor and external devices using 8 bit IN and OUT instructions. The IN instruction takes 8 bits from the input device and stores it in Rd bit position 0-7 with the rest set to 0. The OUT instruction takes from Rs bit position 0-7 and sends them to the output device.

B. Interrupt Driven I/O:

ION: T3: $PSR[I] \leftarrow 1$, $SC \leftarrow 0$

IOF: T3: $PSR[I] \leftarrow 0$, $SC \leftarrow 0$

Explanation:

When $PSR[I]=1$ and a connected device sends a interrupt request the processor completes it's current task and enters interrupt cycle. Here are our interrupts are enabled and disabled using the ION and IOF instructions.

C.Interrupt Cycle

RT0: $AR \leftarrow R7 - 1, R7 \leftarrow R7 - 1$

RT1: $M[AR] \leftarrow PC, PC \leftarrow 0x000$

RT2: $PC \leftarrow PC + 1, PSR[I] \leftarrow 0, PSR[M] \leftarrow 1, SC \leftarrow 0$

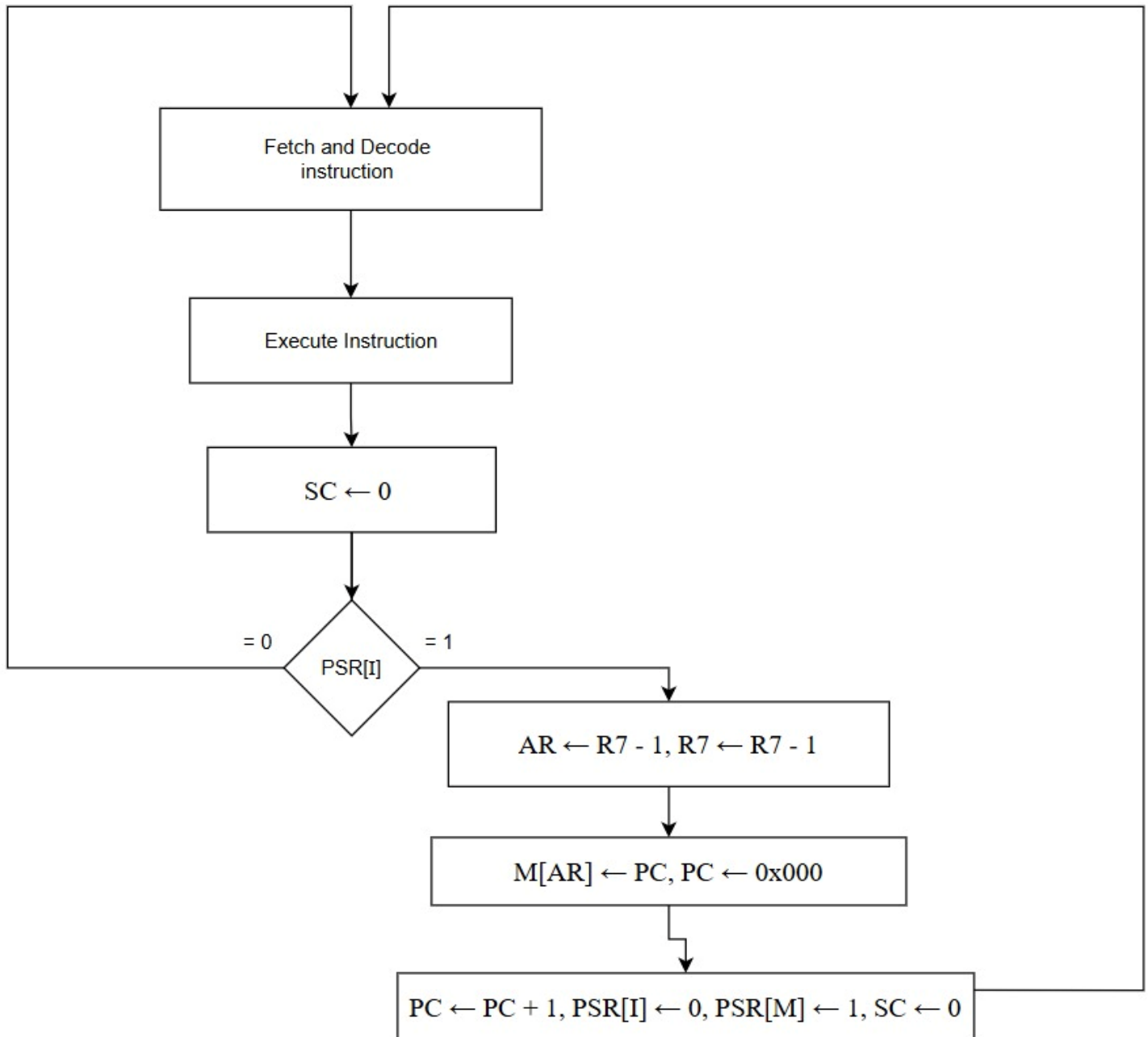


FIGURE 14: Processor Input/Output Configuration

Explanation:

The interrupt cycle is a hardware initiated sequence that saves the current processor state and transfers control to the interrupt service routine. At RT0 the current PC value is saved into a temporary register TR and AR is pointed to address 0x000. At RT1 the return address is stored at memory location 0x000 and PC is set to 0. At RT2 PC increments to 1 where the interrupt service routine address is stored, the I bit is cleared to prevent nested interrupts, and the M bit is set to supervisor mode. The processor then executes the interrupt service routine. When the routine completes it restores the PSR, re-enables interrupts using ION, and returns to the interrupted program by loading the saved PC from address 0x000.

XI. Complete Computer Description

Our processor is a 24-bit stored program computer whose complete design has been presented across the preceding sections. This section ties all components together into a unified system description and summarizes the key innovations our design introduces over the Mano Machine.

The processor consists of the following hardware components working together as a complete system:

Memory Unit : a 4096 location $\times$ 24-bit unified memory serving both program instructions and data storage. All memory access is exclusively routed through the Address Register which is the only component connected to the 12-bit memory address bus.

Register File : eight 24-bit general purpose registers R0 through R7 implemented as a three port structure with two simultaneous read ports and one write port. R0 is permanently zero, R6 serves as the link register for subroutine calls, and R7 serves as the stack pointer initialized at 0xFFFF growing downward through the stack space at addresses 0xF00 to 0xFFF.

Process Status Register : an 8-bit protected register with dedicated connections to the ALU for condition flags Z, N, C and V, and direct connections to the Control Unit for the interrupt enable bit I and supervisor mode bit M.

Arithmetic Logic Unit : a 24-bit combinational circuit consisting of a 24-bit adder-subtractor unit, a logic unit performing AND OR XOR and NOT operations, and a shift unit performing CIL CIR SHL and SHR operations. All three units operate simultaneously and a multiplexer selects the result based on the 4-bit ALU_OP signal from the Control Unit.

Control Unit : a hardwired control unit consisting of a 5 $\times$ 32 instruction decoder, a 2 $\times$ 4 mode decoder with Dual D flip flop, a 3 $\times$ 8 Rd decoder, a 2 $\times$ 4 Rs decoder, a 4-bit sequence counter feeding a 4 $\times$ 16 timing decoder, and combinational control logic gates. The CU generates all 21 control bus signals that coordinate every data movement and operation across the processor.

Common Bus : a 24-bit tri-state common bus connecting all components through three-state buffer gates controlled by selection variables S0, S1 and S2. The IR is the only component not connected to the bus since its fields are routed directly to the CU decode logic through dedicated internal lines.

XII. Bibliography

- [1] M. M. Mano and M. D. Ciletti, *Computer System Architecture*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 1993.
- [2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.
- [3] V. Fang and C. Kohl, “A simulated Mano machine—A novel project for computer architecture class,” in *Proceedings of the American Society for Engineering Education (ASEE) Annual Conference*, 2008.